26
Real-time Gaming Leaderboard

In this chapter, we are going to walk through the challenge of designing a leaderboard for an online mobile game.

What is a leaderboard? Leaderboards are common in gaming and displayed on a screen where a player is ranking a particular leaderboard or multiple leaderboards. Leaderboards are a competitive element that allows players to see how they rank against other players. There are a variety of ways to implement a leaderboard. Factors that influence the ranking of the leading competitors and also displays the position of the user are:

Factors
Kill Count
Time
Points
Bonus Points
Accuracy
Combo Multiplier

Figure 1 Leaderboard

Step 1 - Understand the Problem and Establish Design Scope

Leaderboards can be complicated. Depending on the game and a number of different criteria that can add complexity, the should clarify the requirements with the interviewer.

Candidate: How is the score calculated for the leaderboard?
Interviewer: The user gets a point when they die a match. We get a kill count system in which each user has a score associated with them. Each time the user wins a match, we should add a point to their total score.

Candidate: Do all players included in the leaderboard?
Interviewer: Yes

Candidate: Is there a time segment associated with the leaderboard?
Interviewer: Good question, we want all-time and weekly leaderboards.

Candidate: Can we assume we only care about the top 10 users?
Interviewer: Yes, we should be able to display the top 10 users as well as the position of a specific user on the leaderboard. If they ahead, only also discuss how to return users who are the above above and below a specific user.

Candidate: How many users are there in the game?
Interviewer: Average of 2 million daily active users (DAU) and 20 million monthly active users (MAU).

Candidate: How many matches are played on average during a tournament?
Interviewer: Each player plays 10 matches per day on average.

Candidate: How do we determine the users that will be in a tournament?
Interviewer: It differs for each game. For the game, it's the above 2 million DAU users.

Candidate: Does the leaderboard need to be real-time?
Interviewer: Yes, we want to display the results, or at close as possible. It is not easy to present a stale/old history of results.

We have our all questions answered, let's list the functional requirements.

- Display top 10 players on the leaderboard.
- Show a user's specific rank.
- Display players who are four places above and below the desired user (bonus).

Other less-defining functional requirements, it's important to understand non-functional requirements.

Non-functional requirements:

- Real-time update or scores.
- Score update is reflected in the leaderboard in real-time.
- Service is primarily latency sensitive, not consistency.

Back-of-the-envelope estimation

Let's take a look at some of the envelope calculations to determine the potential scale and challenges our solution will need to address.

With 2 million DAU in the game had an even distribution of players during a 24 hour period, we would have an average of 50 users per second (2,000,000 DAU / 24 hours) = 50). However, we know that usages most likely will cluster around peak times, and we should plan for peak load. Assuming peak load is four times the average load, we should be able to handle 200 users per second. We need to plan and scale/architecture for this.

Assuming each user plays 10 matches per day, we have a peak load of 200 users per second.

QPS for users scoring a point if 4 users play 10 games per day on average, the QPS for users scoring a point is: 50 * 10 = 500. Peak QPS is 5 for the average QPS = 2500.

QPS for fetching the top 10 leaderboard assume a user opens the game once a day and fetches the top 10 leaderboard is loaded only when a user first opens the game. For QPS the load is around 50 QPS.

Step 2 - Propose High-Level Design and Get Buy-In

In this section, we will discuss API design, high-level architecture, and data models.

API design

As a high level, we need the following three APIs:

POST /v1/score

Update a user's position on the leaderboard when a user wins a game. The request parameters are listed below. This should be an internal API and only be accessible by the game servers. The response should be an JSON to update the leaderboard score directly.

Field	Description
user_id	The user who wins a game.
points	The number of points earned by winning a game.

Table 1

Response:

Name	Description
200 OK	Successfully updated a user's score.
400 Bad Request	Failed to update a user's score.

Table 2

GET /v1/leaderboard

Fetch the top 10 players from the leaderboard.

Sample response:

```
{
  "data": [
    {
      "user_id": "user_id1",
      "display_name": "name1",
      "score": "1000",
      "rank": "1"
    },
    {
      "user_id": "user_id2",
      "display_name": "name2",
      "score": "900",
      "rank": "2"
    }
  ],
  "total": 10
}
```

GET /v1/leaderboard/user_id

Fetch the rank of a specific user.

Field	Description
user_id	The id of the user whose rank we would like to fetch.

Table 3

Sample response:

```
{
  "user_id": "user_id",
  "display_name": "name",
  "score": "800",
  "rank": "8",
  "prev": [
    {
      "user_id": "user_id5",
      "display_name": "name5",
      "score": "900",
      "rank": "7",
    }
  ],
  "next": [
    {
      "user_id": "user_id9",
      "display_name": "name9",
      "score": "700",
      "rank": "9",
    }
  ]
}
```

High-level architecture

The high-level design diagram is shown in Figure 2. There are two services in this design. The game service allows users to play the game and the leaderboard service creates and displays a leaderboard.

Figure 2 High-level design

1. When a player wins a game, the client sends a request to the game service.
2. The game service ensures the win is valid and calls the leaderboard service to update the score.
3. The leaderboard service returns the top 10 leaderboard to the client.
4. A player makes a call to the leaderboard service to fetch their leaderboard position. Including:
   a. Top 10 leaderboard.
   b. the rank of the player on the leaderboard.

Before setting on this design, we considered a few alternatives and decided against them. It might be helpful to go through the thought process of the architect between different options.

Should the client talk to the leaderboard service directly?

Figure 3 Client-server and alternative option

In the alternative design, the score is sent to the client. The score can be tampered, so it is subject to man-in-the-middle attack (or), client-side code can be reverse engineered, so it is subject to a client-side attack. Thus, we should not use this design.

Note that for some architectural patterns such as online poker, the client may not need to call the game server explicitly to set scores. The game server and client talk to each other, and it knows when the game finishes and can update the score publicly. As this is an in-depth requirement based on the conversation with the interviewer, we do not use a message queue in our design.

Figure 4 Game score are sent by multiple services

Data models

The only data component in the system is the leaderboard store. We will discuss three potential solutions: relational database, Redis, and NoSQL. Ultimately, Redis is selected in deep dive.

Relational database solution

First, let's take a deep look and select the relevant solution. Let's start with a relational database. What do we need?

We would most likely be able to create leaderboard specific tables in a relational database system (RDS). Such monthly leaderboard could be represented as a database table containing user id and score columns. When the user wins a match, either assert the user if it does not exist in the table or update the score if the user is already in the leaderboard. Finally, the leaderboard can query and return the top 10 rows in descending order. This details are acceptable here.

Leaderboard DB table:

leaderboard

user_id	varchar
score	int

Figure 5 Leaderboard table

In reality, the leaderboard table has additional information, such as a game_id, a timestamp, etc. However, the underlying logic of how to update and fetch the leaderboard remains the same. For simplicity, we assume only the current month's leaderboard data is stored in the leaderboard table.

A user wins a point:

Figure 6 A user wins points

Assume every score update would be an increment of 1 if a user doesn't yet have an entry in the leaderboard for the month, the first point scored.

```
INSERT INTO leaderboard (user_id, score) VALUES ('mary1924', 1);
```

An update to the user's score would be:

```
UPDATE leaderboard set score=score + 1 where user_id='mary1924';
```

Find a user's leaderboard position:

Figure 7 Fetch a user's leaderboard position

To fetch the user rank, we could select the leaderboard table and rank by the score.

```
SELECT rankquery.* FROM (SELECT DB.user_id, DB.score,
RANK() OVER
ORDER BY score DESC)
```

The result of the SQL query looks like the following:

rank	user_id	score
1	mary_kiss9	987
2	mathew	900
3	sarah	870
4	mary1924	850

Table 4 Result ordering by score

This solution works when the data set is small, but the query becomes very slow when there are millions of rows. Let's take a look at why.

To figure out the rank of a user, we need to sort every single player into their correct spot on the leaderboard so we can determine their position. Sorting is the most time-consuming operation because it requires disk I/O operations and uses a lot of CPU and RAM if the data set is large.

SQL databases are not performant when we have to process large amounts of continuously changing information. Attempting to do a real-time update on the leaderboard for 50 users per second would be challenging. Not only would the leaderboard be constantly locked because the ranking is changing, but some of the queries would take longer than a second.

A relational database is not designed to handle the high level of query volume this implementation would require. An RDS could be used successfully if there were no requirements for real-time updates, but that would not be a requirement in terms of our design for the leaderboard.

One optimization we can do is to add an index and limit the number of pages to scan with the LIMIT clause. The query looks like this:

```
SELECT DB.rankq, * FROM (SELECT DB.user_id, DB.score,
RANK() OVER
ORDER BY score DESC)
LIMIT 0,1000;
```

However, this approach doesn't scale well. Time, finding a user's rank is a lot performant because it essentially requires a table scan to determine the rank. Second, this approach doesn't provide a straightforward solution for determining the rank of a user who is not at the top of the leaderboard.

Redis solution

We want to find a solution that gives us predictable performance even for millions of users and allows us to have easy access to common leaderboard operations, whether needing to fetch per or complex ranking.

Redis provides a persistent solution to in-memory, Redis is an in-memory data store supporting key-value pairs. Since it works in memory, it can be very fast and performant. Redis has a specific **Sorted Set** type that is ideal for building leaderboard-type design problems.

What are sorted sets?

A sorted set is a unique data kind in set. Each member of a sorted set is associated with a score. The members of a set must be unique, but the score may be duplicate.

Our leaderboard use case is supported by sorted sets. Member in a leaderboard set is implemented by two data structures: a hash table and a skip list [1]. The hash table maps users to scores and the skip list maps scores to users. In sorted sets, users are ordered by scores. A user's position on the leaderboard can be found in a sorted set in a hash table with complexity of O(1). As mentioned in Figure 8, the table is sorted by score in descending order.

Figure 8 Redis leaderboard is represented by a sorted set

In this chapter, we don't go into the full detail of how Redis sorted sets are implemented by two data structures, a hash table and a skip list [1]. The hash table maps users to scores and the skip list maps scores to users. In sorted sets, users are ordered by scores. A user's position on the leaderboard can be found in a sorted set in a hash table with complexity of O(1). As mentioned in Figure 8, the table is sorted by score in descending order.

Figure 9 The skip list with a tower of towers

When the data set is small, the speed improvement using the skip list isn't obvious. Figure 10 shows an example of a skip list with 3 levels of towers. In the base-level list, it needs to travel 10 nodes to reach the correct node. In the skip list, it only needs to traverse 7 nodes [2].

Figure 10 Skip list with 3 levels of towers

Sorted sets are more performant than a relational database because retrieval is automatically positioned in the right order during insertion, and the rank can be found with the complexity of an add or find operation in a sorted set in logarithm. O(log(n)).

In contrast, to calculate the rank of a specific user in a relational database, we need to run ranked queries:

```
SELECT * FROM(SELECT COUNT(*) FROM leaderboard ?GT
WHERE score > ?score) AS GT,
COUNT(*) FROM leaderboard ?EQ
WHERE score = ?score) AS EQ, ...LIMIT ...
```

Redis provides a simple command to calculate a user's rank by the score. We can specify the rank (single or, converge), the number of entry, and the position (reverse). The data Diagram shows the data, where is the number of entries to fetch (which is usually small 4 or case), and x is the number of entries in the sort set.

ZREVRANGEBYRANK: Find the position of an key using lexicographical order in lexemes: Info.

Workflow with sorted sets

1. Jan scores a point:

Figure 11 Jan scores a point

Every month we create a new leaderboard sort and the previous years are moved to historical data storage. When a user wins a match, they score 1 point, so we call ZINCRBY to increment the user's score by 1 in that month's leaderboard, or add the user to the leaderboard if they don't exist yet. Redis will return the new score.

```
ZINCRBY mary_kiss9 1 "mary1924"
```

The following command adds a point to user mary1924 after they win a match.

```
ZINCRBY leaderboard_key_2021 1 "mary1924"
```

2. Jan fetches his top 10 global leaderboard

Figure 12 Jan fetches his top 10 global leaderboard

We will call ZREVRANGE to obtain the members in descending order because we want the highest scores, and pass the "WITHSCORES" attribute to ensure that it also returns the total score for each user, as well as the set of users with the highest scores. The following command fetches the top 10 users with the highest scores.

```
ZREVRANGE leaderboard_key_2021 0 9 WITHSCORES
```

This returns a list like this:

```
[user1,score1],[user2,score2],[user3,score3], ...]
```

The result of the SQL query looks like the following:

Figure 13 Find a user's leaderboard position

To fetch the position of a user in the leaderboard, we call ZREVRANK with the user's name on the leaderboard. Again, we call the sort version of the command because we want to score from high to low.

```
ZREVRANK leaderboard_key_2021 "mary1924"
```

4. Fetch the relative position in the leaderboard for a user. An example is shown in Figure 14.

Rank	User	Points
207	Aquaboss	870
208	Jr Super	869
209	Bishop's Angels	867
210	Nightwolf	866
211	Nebula007	865
212	Mike78	740
213	Hawk1119	739
214	Ugly<<<<<<	738
215	Mister_K	737

Figure 14 Rank 4 places above and below

While not an explicit requirement, we can easily fetch the relative rank. For example, by leveraging ZREVRANGE with the number of results above and below the desired point. For example, if user mary1924's rank is 207 and we want to fetch 4 players above and below, we would call the following command:

```
ZREVRANGE leaderboard_key_2021 207 215
```

Storage requirement

At a minimum, we need to store the user id and points. The total size estimate is roughly 24 million monthly active users times size of two integers (8 bytes) = 384 MB. Assuming we need to keep the data for the past 10 years, the total storage requirement is 46 GB. Assuming 10 years, we need 30 terabyte of storage and replication factor. Given the worst-case scenario of one leaderboard every day (MAU, we need latest 30 years) 1.2 million x 300 million factor or ~350 MB for leaderboard storage in the Redis cache. Even if we double the size, it is still a small cache. Redis is a good fit for our use case, and we can keep the current year's leaderboard in Redis cache and the rest in persistent storage.

Another added Redis benefit is persistence. Redis can persist data to disk. The data will last if the envelope estimation is 350 operations. This is an extra-time-consumption of a single Redis server.

One concern about the Redis cache is persistence, as a Redis node might fail. Luckily, Redis does support persistence, but rebuilding a cache from persistent storage is slow. To ensure high availability, we can configure a primary-replica cluster, and when the primary node is unhealthy, we can a new replica promote.

Besides, we need to have 2 supporting tables (user and points) in a relational database like MySQL. The user table would store the user ID and name to display the user in leaderboard applications. We would store the points table. The point table would store the user ID, points, and timestamp. We need the points table to recover the leaderboard if the cache server fails, and we can use the data to recreate the Redis leaderboard. We do not need to store all of the leaderboard.

It is a good communication strategy to map tables similar to cases, so people can understand the design. Specifically for this, it's always clear how we retrieve most frequently. However, this doesn't prevent a large amount of data.

Step 3 - Design Deep Dive

Now that we discussed the high-level design, let's dive into the following:

- Whether or not to scale Redis horizontally.
  - Manage our cron services.
  - Leverage cloud service providers like Amazon Web Services (AWS).
- Scaling Redis.
- Alternative solution: NoSQL.
- Other considerations.

To use a cloud provider or not

Depending on the existing infrastructure, we generally have two options for deploying our solution. Let's take a look at each of them.

Manage our own services

In this approach, we will create a leaderboard server and each cronjob to store the leaderboard data for that period. The cronset set stores member and score information. The set of the details about each cronset, such as start date and end date stages, are stored in Redis database. When fetching the leaderboard, handle the leaderboard data, API service asks the database to fetch corresponding cronset information. The API service then fetches the leaderboard data from the cronset. If a user's score changes, the API service will update its own cronset as well.

Client
API server
Leaderboard Cronset
Redis database
Cronset
Cronset
Cronset

Figure 17 Retrieve leaderboard

Lambdas are great because they are a serverless approach, and the infrastructure will take care of auto-scaling the function as needed. This means we don't need to manage the scaling and environment setup and maintenance. Given this, we recommend going with a serverless approach if we build the game from the ground up.

Scaling Reds

With 5 million DAU, we can get away with one Redis cache from both a storage and QPS perspective. However, let's imagine we have 500 million DAU, which is 100 times our original scale. Now our worst-case scenario for the size of the leaderboard goes up to 65 GB (650 MB *100), and our QPS goes up to 250,000 (2,500 * 100) queries per second. This calls for a sharding solution.

Data sharding

We consider sharding in one of the following two ways: fixed or hash partitions.

Fixed partition

One way to understand fixed partitions is to look at the overall range of points on the leaderboard. Let's say that the number of points won in one month ranges from 1 to 1000, and we break up the data by range. For example, we could have 10 shards and each shard would have a range of 100 scores (For example, 1-100, 101-200, 201-300, ...) as shown in Figure 18.

Figure 18 Fixed partition

For this to work, we want to ensure there is an even distribution of scores across the leaderboard. Otherwise, we need to adjust the score range in each shard to make sure of a relatively even distribution. In this approach, we shard the data ourselves in the application code.

When we are inserting or updating the score for a user, we need to know which shard they are in. We could do this by calculating the user's current score from the MySQL database. This can work, but a more performant way would be to create a secondary cache to store the mapping from user ID to score. We need to be careful when a user increases their score and moves between shards. In this case, we need to remove the user from their current shard and move them to the new shard.

To fetch the top 10 players in the leaderboard, we would fetch the top 10 players from the shard (sorted set) with the highest scores. In Figure 18, the last shard with scores (901, 1000) contains the top 10 players.

To fetch the rank of a user, we would need to calculate the rank within their current shard (local rank), as well as the total number of players with higher scores in all of the shards. Note that the total number of players in a shard can be retrieved by running the "*info keyspace*" command in OYD [10].

Hash partition

A second approach is to use the Redis cluster, which is literally the answer if scores are very clustered or clumped. Redis cluster provides a way to shard data automatically across multiple Redis nodes. It doesn't use consistent hashing but a different form of sharding, where every key is part of a **hash slot**. There are 16384 hash slots [11] and we can compute the hash slot of a given key by doing CRC16(key) % 16384 [12]. This allows us to add and remove nodes in the cluster easily without redistributing all the keys. In Figure 19, we have 3 nodes, where:

- The first node contains hash slots [0, 5500].
- The second node contains hash slots [5501, 11000].
- The third node contains hash slots [11001, 16383].

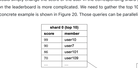

Figure 19 Hash partition

An update would simply change the score of the user in the corresponding shard (determined by CRC16(key)%16384). Retrieving the top 10 players on the leaderboard is more complicated. We need to gather the top 10 players from each shard and have the application sort the data. A concrete example is shown in Figure 20. Those queries can be parallelized to reduce latency.

Figure 20 Scatter-gather

This approach has a few limitations:

- When we need to return top K results (where K is a very large number) on the leaderboard, the latency is high because a lot of entries are returned from each shard and need to be sorted.
- Latency is high if we have lots of partitions because the query has to wait for the slowest partition.
- Another issue with this approach is that it doesn't provide a straightforward solution for determining the rank of a specific user.

Therefore, we lean towards the first proposal: fixed partition.

Sizing a Redis node

There are multiple things to consider when sizing the Redis nodes [13]. Write-heavy applications require much more available memory, since we need to be able to accommodate all of the writes to create the snapshot in case of a failure. To be safe, allocate twice the amount of memory for write-heavy applications.

Redis provides a tool called Redis-benchmark that allows us to benchmark the performance of the Redis setup, by simulating multiple clients executing multiple queries and returning the number of requests per second for the given hardware. To learn more about Redis-benchmark, see [14].

Alternative solution: NoSQL

An alternative solution to consider is NoSQL databases. What kind of NoSQL should we use? Ideally, we want to choose a NoSQL that has the following properties:

- Optimized for writes.
- Efficiently sort items within the same partition by score.

NoSQL databases such as Amazon's DynamoDB [15], Cassandra, or MongoDB can be a good fit. In this chapter, we use DynamoDB as an example. DynamoDB is a fully managed NoSQL database that offers reliable performance and great scalability. To allow efficient access to data with attributes other than the primary key, we can leverage global secondary indexes [16] in DynamoDB. A global secondary index contains a selection of attributes from the parent table, but they are organized using a different primary key. Let's take a look at an example.

The updated system diagram is shown in Figure 21. Redis and MySQL are replaced with DynamoDB.

Figure 21 DynamoDB solution

Assume we design the leaderboard for a chess game and our initial table is shown in Figure 22. It is a denormalized view of the leaderboard and user tables and contains everything needed to render a leaderboard.

Primary key		Attributes		
user_id	score	profile_pic	leaderboard_name	
lovelace	300	lova@leet.com	https://cdn.example/1.png	chess#2020-02
i_love_tofu	300	tao@leet.com	https://cdn.example/a.png	chess#2020-02
golden_gate	100	grit@leet.com	https://cdn.example/g.png	chess#2020-03
pizza_or_bread	300	pb@leet.com	https://cdn.example/1.png	chess#2021-05
ocean	10	ocea@leet.com	https://cdn.example/10.png	chess#2020-02
...	...	...	...	...

Figure 22 Denormalized view of the leaderboard and user tables

This table scheme works, but it doesn't scale well. As more rows are added, we have to scan the entire table to find the top scores.

To avoid a linear scan, we need to add indexes. Our first attempt is to use "*game_name#year-month*" as the partition key and the score as the sort key, as shown in Figure 23.

Global Secondary Index Partition key (PK)	Sort key (SK) [score]	Attributes user_id	email	profile_pic
chess#2020-02	300	lovelace	lova@leet.com	https://cdn.example/1.png
chess#2020-02	209	i_love_tofu	tao@leet.com	https://cdn.example/a.png
chess#2020-03	100	golden_gate	grit@leet.com	https://cdn.example/2.png
chess#2020-02	300	pizza_or_bread	pb@leet.com	https://cdn.example/1f.png
chess#2020-02	10	ocean	ocea@leet.com	https://cdn.example/10.png
...	...	...	...	...

Figure 23 Partition key and sort key

This works, but it runs into issues at a high load. DynamoDB splits data across multiple nodes using consistent hashing. Each item lives in a corresponding node based on its partition key. We want to structure the data so that data is evenly distributed across partitions. In our table design (Figure 23), all the data for the most recent month would be stored in one partition and that partition becomes a hot partition. How can we solve this problem?

We can split data into *N* partitions and append a partition number (user_id % number_of_partitions) to the partition key. This pattern is called write sharding. Write sharding adds complexity for both read and write operations, so we should consider the trade-offs carefully.

The second question we need to answer is, how many partitions should we have? It can be based on write volume or DAU. The important thing to remember is that there is a trade-off between load on partitions and read complexity. Because data for the same month is spread evenly across multiple partitions, the load for a single partition is much lighter. However, to read items for a given month, we have to query all the partitions and merge the results, which adds read complexity.

The partition key looks something like this: "*game_name#year-month#partition_number0*". Figure 24 shows the updated schema table.

Global Secondary Index Partition key (PK)	Sort key (SK) [score]	Attributes user_id	email	profile_pic
chess#2020-02#0	300	lovelace	lova@leet.com	https://cdn.example/1.png
chess#2020-02#0	209	i_love_tofu	tao@leet.com	https://cdn.example/a.png
chess#2020-03#0	100	golden_gate	grit@leet.com	https://cdn.example/2.png
chess#2020-02#0	300	pizza_or_bread	pb@leet.com	https://cdn.example/1f.png
chess#2020-02#0	10	ocean	ocea@leet.com	https://cdn.example/10.png
...	...	...	...	...

Figure 24 Updated partition key

The global secondary index uses "*game_name#year-month#partition_number*" as the partition key and the score as the sort key. What we end up with are N partitions that are all sorted within their own partition (locally sorted). If we assume we had 3 partitions, then in order to fetch the top 10 leaderboard, we would use the approach called "scatter-gather" mentioned earlier. We would fetch the top 10 results in each of the partitions (this is the "scatter" portion), and then we would allow the application to sort the results among all the partitions (this is the "gather" portion). An example is shown in Figure 25.

Figure 25 Scatter-gather

How do we decide on the number of partitions? This might require some careful benchmarking. More partitions decrease the load on each partition but add complexity, as we need to scatter across more partitions to build the final leaderboard. By employing benchmarking, we can see the trade-off more clearly.

However, similar to the Redis partition solution mentioned earlier, this approach doesn't provide a straightforward solution for determining the relative rank of a user. But it is possible to get the percentile of a user's position, which could be good enough. In real life, telling a player that they are in the top 10-20% might be better than showing the exact rank at eg.1,000,001. Therefore, if the scale is large enough that we needed to shard, we could assume that the score distributions are roughly the same across all shards. If this assumption is true, we could have a cron job that analyzes the distribution of the score for each shard, and caches that result.

The result would look something like this:

```
10th percentile = score = 100
20th percentile = score = 500

90th percentile = score = 6500
```

Then we could quickly return a user's relative ranking (say 90th percentile).

Step 4 - Wrap Up

In this chapter, we have created a solution for building a real-time game leaderboard with the scale of millions of DAU. We explored the straightforward solution of using a MySQL database and realized that approach because it does not scale to millions of users. We then designed the leaderboard using Redis sorted sets. We also looked into scaling this solution to 500 million DAU, by leveraging sharding across different Redis caches. We also proposed an alternative NoSQL solution.

In the event you have some extra time at the end of the interview, you can cover a few more topics:

Faster retrieval and breaking tie

A Redis Hash provides a map between string fields and values. We could leverage a hash for 2 use cases:

1. To store a map of the user id to the user object that we can display on the leaderboard. This allows for faster retrieval than having to go to the database to fetch the user object.
2. In the case of two players having the same scores, we could rank the users based on who received that score first. When we increment the score of the user, we can also store a map of the user id to the timestamp of the most recently won game. In the case of a tie, the user with the older timestamp ranks higher.

System failure recovery

The Redis cluster can potentially experience a large-scale failure. Given the design above, we could create a script that leverages the fact that the MySQL database records an entry with a timestamp each time a user wins a game. We could iterate through all of the entries for each user, and call ZINCRBY once per entry, per user. This would allow us to recreate the leaderboard offline if necessary, in case of a large-scale outage.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Chapter Summary

Reference Material

[1] Redis Sorted Set source code:
https://github.com/redis/redis/blob/unstable/src/t_zset.c

[2] Geekbang:
https://time.geekbang.org/resourcedownload?id=462948&fid=231301287153135vOu7fcdPba8qaS.jpg

[3] Building real-time Leaderboard with Redis:
https://medium.com/@handeep/5-eternal-building-real-time-leaderboard-with-redis-82c98ae47f3f

[4] Build a real-time gaming leaderboard with Amazon ElastiCache for Redis:
https://aws.amazon.com/blogs/database/building-a-real-time-gaming-leaderboard-with-amazon-elasticache-for-redis

[5] How we created a real-time Leaderboard for a million Users:
https://medium.com/gamesquad/how-we-created-a-real-time-leaderboard-for-a-million-users-555eacb2cf7b

[6] Leaderboards: https://redislabs.com/solutions/use-cases/leaderboards/

[7] Lambda: https://aws.amazon.com/lambda/

[8] Google Cloud Functions: https://cloud.google.com/functions

[9] Azure Functions: https://azure.microsoft.com/en-us/services/functions/

[10] Info command: https://redis.io/commands/info/?

[11] Why redis cluster only have 16384 slots:
https://yarlson.ffiw.com/questions/38203332/why-redis-cluster-only-have-16384-slots

[12] Cyclic redundancy check: https://en.wikipedia.org/wiki/Cyclic_redundancy_check

[13] Choosing your node size:
https://docs.aws.amazon.com/amazonElastiCache/latest/red-ug/nodes-select-size.html

[14] How fast is Redis?: https://redis.io/topics/benchmarks

[15] Using Global Secondary Indexes in DynamoDB:
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GSI.html

[16] Leaderboard & Write Sharding: https://www.dynamodbguide.com/leaderboard-write-sharding/

Mark as Complete

Copyright ©2022-2026 BytebyteGo Inc. All rights reserved.